

BST Distanze

Argomenti: programmazione OOP, IO file, operazioni con le stringhe, BST.

Si implementi una classe BST, attraverso l'utilizzo dei template, contenente elementi di tipo generico. La classe dovrà implementare la procedura di inserimento di un nuovo elemento, la procedura di cancellazione ed una funzione che, prese in input due chiavi k e h, sia in grado di calcolare la distanza tra i nodi che contengono k e h rispettivamente.

La classe BST può essere implementata da zero, oppure si può estendere arbitrariamente la definizione implementata nel file `bstT.cpp` fornito, aggiungendo i metodi mancanti.

L'input è suddiviso in 6 task, uno per ogni riga, specificato in un file di testo *inputDistanze.txt*. Ogni riga del file di input è formata da 3+N elementi. Il primo elemento è una stringa che identifica il tipo di dato che dovrà essere contenuto all'interno dell'albero. Il secondo elemento è un valore intero N, il quale rappresenta il numero delle operazioni di inserimento e cancellazione che dovranno essere effettuate nella struttura dati. Seguono N stringhe che rappresentano, nell'ordine dato, le operazioni da svolgere all'interno dell'albero. Nell' specifico un inserimento di un valore "v" viene identificato dalla stringa "ins:" seguita dal valore "v". In modo simile la cancellazione di un valore "v" viene identificato dalla stringa "canc:" seguita dal valore "v". Gli ultimi due elementi della sequenza sono i valori k ed h, i quali rappresentano le chiavi contenute nei nodi per i quali si vuole calcolare la distanza.

Alcune righe di esempio:

```
int 10 ins:80 ins:172 ins:13 ins:138 ins:223 ins:245 ins:144 ins:152 ins:151 ins:270 151 138
int 10 ins:54 ins:285 ins:218 ins:95 ins:166 ins:272 ins:118 ins:124 ins:141 ins:84 54 218
int 10 ins:282 ins:208 ins:239 ins:120 ins:37 ins:258 ins:298 ins:141 ins:14 ins:26 14 282
```

Output corrispondenti: 3 2 4

Nota: il codice deve essere implementato in modo tale da poter eseguire la compilazione ed il building su una macchina generica con il comando `g++`. Implementazioni “sofisticate” comporteranno altrettante capacità di utilizzo del programma `g++` da parte dello studente che le propone.